

Meetup Python Bordeaux

Marimo: Python Notebooks to Replace Jupyter Notebooks?

Jean-Luc CHARLES

Jean-Luc.Charles @ mailo.com

17 juin 2025



Presentation Summary

- Jupyter notebooks pros/cons
- Marimo notebook main aspects
- Marimo examples:
 - ~> Reactivity, UI Interactivity with plots
 - ~> Machine Learning
 - ~> Machine Learning with disk cache.



Ressources

- GitHub repository for the PDF slides and the notebooks:
github.com/cjlux/marimo_notebooks



Who am I



Season 1

- Industrial career of ~10 years
Signal Processing at Thomson-SINTRA then OO design & C++ development at IT companies (Capgemini...)
- I joined the I2M lab of ENSAM (Bordeaux) as lecturer
I taught Scientific & OO programming in Python at ENSAM.
- I started to work on AI & Deep Learning in 2015.

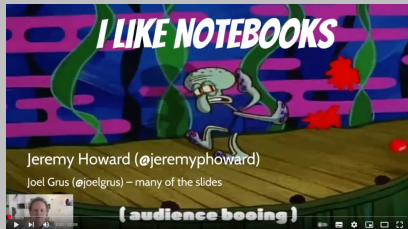


Season 2

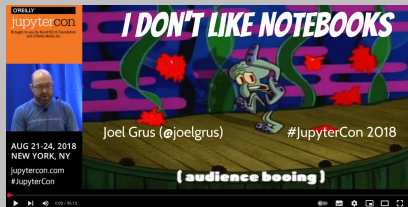
- Still working on *AI / Data processing* with Python as a freelancer.

Jupyter notebooks *pros/cons*

Some people like working with Jupyter notebooks... [[How20](#)]

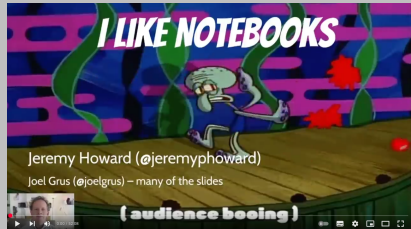


and some don't [[Gru18](#)]

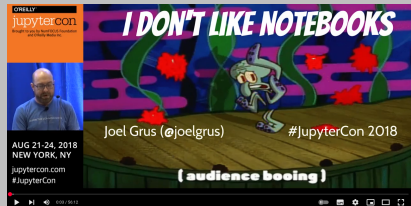


Jupyter notebooks *pros/cons*

Some people like working with Jupyter notebooks... [[How20](#)]



and some don't [[Gru18](#)]



What are the main *pros/cons* of Jupyter Notebooks...

Jupyter notebooks *pros/cons*



Some advantages

- Very well known...
- A very large number of published notebooks, covering very varied application areas
- Huge user community, a lot of tutos
- Powerfull to create documentation with embedded graphs, pictures...
- Nicely permissive: interactive asynchronous execution of cells
- and many more...

Jupyter notebooks *pros/cons*



Some advantages

- Very well known...
- A very large number of published notebooks, covering very varied application areas
- Huge user community, a lot of tutos
- Powerfull to create documentation with embeded graphs, pictures...
- Nicely permissive: interactive asynchronous execution of cells
- and many more...

Jupyter notebooks *pros/cons*



Some advantages

- Very well known...
- A very large number of published notebooks, covering very varied application areas
- Huge user community, a lot of tutos
- Powerfull to create documentation with embeded graphs, pictures...
- Nicely permissive: interactive asynchronous execution of cells
- and many more...

Jupyter notebooks *pros/cons*



Some advantages

- Very well known...
- A very large number of published notebooks, covering very varied application areas
- Huge user community, a lot of tutos
- Powerfull to create documentation with embeded graphs, pictures...
- Nicely permissive: interactive asynchronous execution of cells
- and many more...

Jupyter notebooks *pros/cons*



Some advantages

- Very well known...
- A very large number of published notebooks, covering very varied application areas
- Huge user community, a lot of tutos
- Powerfull to create documentation with embeded graphs, pictures...
- Nicely permissive: interactive asynchronous execution of cells
- and many more...

Jupyter notebooks *pros/cons*

😊 Some advantages

- Very well known...
- A very large number of published notebooks, covering very varied application areas
- Huge user community, a lot of tutos
- Powerfull to create documentation with embeded graphs, pictures...
- Nicely permissive: interactive asynchronous execution of cells
😊
- and many more...

Jupyter notebooks *pros/cons*

😊 Some advantages

- Very well known...
- A very large number of published notebooks, covering very varied application areas
- Huge user community, a lot of tutos
- Powerfull to create documentation with embeded graphs, pictures...
- Nicely permissive: interactive asynchronous execution of cells
😊
- and many more... 🌟😊

Jupyter notebooks *pros/cons*



Some disadvantages...

- **Hidden state** if you delete a cell, its variables stay in memory, which other cells may still reference $\leadsto$ reproducibility issues.
- **Lack of reproducibility** the code you see doesn't necessarily match the outputs on the page or the program state.
- One study [[Guz20](#)] analyzed 10 million Jupyter notebooks and found that 36% of them weren't reproducible.
- Notebooks JSON format embeds binary outputs (plots, images...) $\leadsto$ Git versioning is difficult
- and even more...

Jupyter notebooks *pros/cons*



Some disadvantages...

- **Hidden state** if you delete a cell, its variables stay in memory, which other cells may still reference $\leadsto$ reproducibility issues.
- **Lack of reproducibility** the code you see doesn't necessarily match the outputs on the page or the program state.
- One study [[Guz20](#)] analyzed 10 million Jupyter notebooks and found that 36% of them weren't reproducible.
- Notebooks JSON format embeds binary outputs (plots, images...) $\leadsto$ Git versioning is difficult
- and even more...

Jupyter notebooks *pros/cons*

☹️ Some disadvantages...

- **Hidden state** if you delete a cell, its variables stay in memory, which other cells may still reference $\leadsto$ reproducibility issues.
- **Lack of reproducibility** the code you see doesn't necessarily match the outputs on the page or the program state.
- One study [[Guz20](#)] analyzed 10 million Jupyter notebooks and found that 36% of them weren't reproducible.
- Notebooks JSON format embeds binary outputs (plots, images...) $\leadsto$ Git versioning is difficult
- and even more...

Jupyter notebooks *pros/cons*

☹️ Some disadvantages...

- **Hidden state** if you delete a cell, its variables stay in memory, which other cells may still reference $\leadsto$ reproducibility issues.
- **Lack of reproducibility** the code you see doesn't necessarily match the outputs on the page or the program state.
- One study [[Guz20](#)] analyzed 10 million Jupyter notebooks and found that 36% of them weren't reproducible.
- Notebooks JSON format embeds binary outputs (plots, images...) $\leadsto$ Git versioning is difficult
- and even more...

Jupyter notebooks *pros/cons*

☹️ Some disadvantages...

- **Hidden state** if you delete a cell, its variables stay in memory, which other cells may still reference $\leadsto$ reproducibility issues.
- **Lack of reproducibility** the code you see doesn't necessarily match the outputs on the page or the program state.
- One study [[Guz20](#)] analyzed 10 million Jupyter notebooks and found that 36% of them weren't reproducible.
- Notebooks JSON format embeds binary outputs (plots, images...) $\leadsto$ Git versionning is difficult
- and even more...

Jupyter notebooks *pros/cons*

☹️ Some disadvantages...

- **Hidden state** if you delete a cell, its variables stay in memory, which other cells may still reference $\leadsto$ reproducibility issues.
- **Lack of reproducibility** the code you see doesn't necessarily match the outputs on the page or the program state.
- One study [[Guz20](#)] analyzed 10 million Jupyter notebooks and found that 36% of them weren't reproducible.
- Notebooks JSON format embeds binary outputs (plots, images...) $\leadsto$ Git versionning is difficult
- and even more... 🤔

Marimo notebooks: pure Python



docs.marimo.io

"marimo notebook: a reinvention of the Jupyter notebook as a reproducible, reactive, interactive, and shareable Python program that can be executed as scripts or deployed as interactive web apps"

Marimo notebooks: pure Python



docs.marimo.io

"marimo notebook: a reinvention of the Jupyter notebook as a reproducible, reactive, interactive, and shareable Python program that can be executed as scripts or deployed as interactive web apps"



Advantages...

- Only pure Python code in a marimo notebook!
 - ~ Cell outputs are not stored in the notebook
- Cells dependency is defined as a DAG (Directed Acyclic Graph)
 - ~ At notebook startup all cells are executed to give the outputs, in the order of their dependencies, not the order they appear.

Marimo notebooks: pure Python



docs.marimo.io

"marimo notebook: a reinvention of the Jupyter notebook as a reproducible, reactive, interactive, and shareable Python program that can be executed as scripts or deployed as interactive web apps"



Advantages...

- Only pure Python code in a marimo notebook!
 - ~ Cell outputs are not stored in the notebook
- Cells dependency is defined as a DAG (Directed Acyclic Graph)
 - ~ At notebook startup all cells are executed to give the outputs, in the order of their dependencies, not the order they appear.

Marimo notebooks: pure Python

Notebooks/simple.py

Simple marimo notebook to show the content of the Python file

```
1 import numpy as np
```

```
1 x = np.arange(1,5)
```

```
array([1, 2, 3, 4])
```

```
1 x
```

```
1 import marimo as mo
```

 PYTHON

 MARKDOWN

 SQL

 GENERATE WITH AI

Marimo notebooks: pur

Notebooks/sin

Simple marimo notebook to show the content of the Py

```
1 import numpy as np
```

```
1 x = np.arange(1,5)
```

```
array([1, 2, 3, 4])
```

```
1 x
```

```
1 import marimo as mo
```

PYTHON

MARKDOWN

```

1 import marimo
2
3 __generated_with = "0.13.15"
4 app = marimo.App(width="medium")
5
6
7 @app.cell(hide_code=True)
8 def _(mo):
9     mo.md(r"""Simple marimo notebook to show the content of the Python file
10     return
11
12
13 @app.cell
14 def _():
15     import numpy as np
16     return (np,)
17
18
19 @app.cell
20 def _(np):
21     x = np.arange(1,5)
22     return (x,)
23
24
25 @app.cell
26 def _(x):
27     x
28     return
29
30
31 @app.cell
32 def _():
33     import marimo as mo
34     return (mo,)
35
36
37 if __name__ == "__main__":
38     app.run()
39

```

Marimo notebooks: pure Python



docs.marimo.io

"marimo notebook: a reinvention of the Jupyter notebook as a reproducible, reactive, interactive, and shareable Python program that can be executed as scripts or deployed as interactive web apps"



Advantages...

- Only pure Python code in a marimo notebook!
 - ~ Cell outputs are not stored in the notebook
- Cells dependency is defined as a DAG (Directed Acyclic Graph)
 - ~ At notebook startup all cells are executed to give the outputs, in the order of their dependencies, not the order they appear.

Marimo notebooks: pure Python



docs.marimo.io

"marimo notebook: a reinvention of the Jupyter notebook as a reproducible, reactive, interactive, and shareable Python program that can be executed as scripts or deployed as interactive web apps"



Advantages...

- Only pure Python code in a marimo notebook!
 - ~> Cell outputs are not stored in the notebook
- Cells dependency is defined as a DAG (Directed Acyclic Graph)
 - ~> At notebook startup all cells are executed to give the outputs, in the order of their dependencies, not the order they appear.

Marimo notebooks: pure Python



docs.marimo.io

"marimo notebook: a reinvention of the Jupyter notebook as a reproducible, reactive, interactive, and shareable Python program that can be executed as scripts or deployed as interactive web apps"



Advantages...

- Only pure Python code in a marimo notebook!
 - ~> Cell outputs are not stored in the notebook
- Cells dependency is defined as a DAG (Directed Acyclic Graph)
 - ~> At notebook startup all cells are executed to give the outputs, in the order of their dependencies, not the order they appear.

Marimo notebooks: pure Python

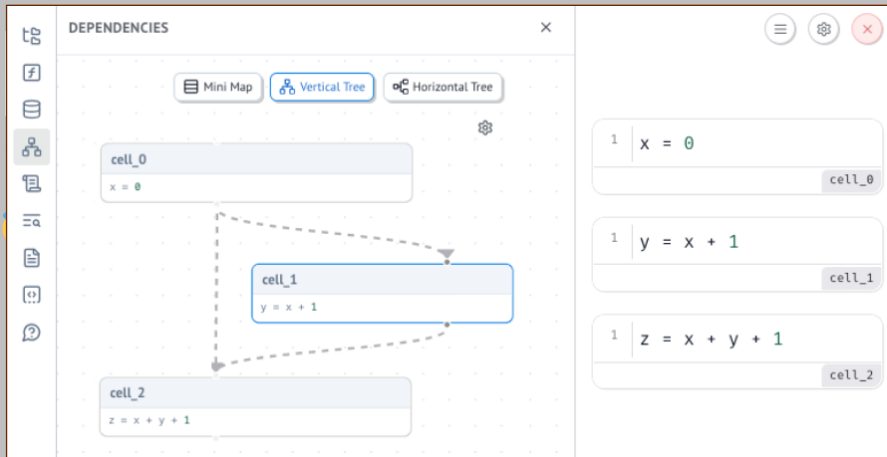


image from [\[Vey24\]](#)

Marimo notebooks: pure Python



docs.marimo.io

"marimo notebook: a reinvention of the Jupyter notebook as a reproducible, reactive, interactive, and shareable Python program that can be executed as scripts or deployed as interactive web apps"



Advantages...

- Only pure Python code in a marimo notebook!
 - ~> Cell outputs are not stored in the notebook
- Cells dependency is defined as a DAG (Directed Acyclic Graph)
 - ~> At notebook startup all cells are executed to give the outputs, in the order of their dependencies, not the order they appear.

Marimo notebooks: pure Python



Advantages...

- **reactive**

- ~ run a cell: marimo reactively runs all dependent cells or marks them as *stale*.
- ~ delete a cell: marimo scrubs its variables from program memory, no hidden state.

- **Consistent state**: notebook code, outputs, and program state are guaranteed to be consistent.

- **Reproducible**: no hidden state, deterministic execution. Notebooks output is always up to date.

Marimo notebooks: pure Python



Advantages...

- **reactive**

- ↪ run a cell: marimo reactively runs all dependent cells or marks them as *stale*.

- ↪ delete a cell: marimo scrubs its variables from program memory, no hidden state.

- **Consistent state**: notebook code, outputs, and program state are guaranteed to be consistent.

- **Reproducible**: no hidden state, deterministic execution. Notebooks output is always up to date.

Marimo notebooks: pure Python



Advantages...

- **reactive**

- ~> run a cell: marimo reactively runs all dependent cells or marks them as *stale*.
- ~> delete a cell: marimo scrubs its variables from program memory, no hidden state.

- **Consistent state**: notebook code, outputs, and program state are guaranteed to be consistent.

- **Reproducible**: no hidden state, deterministic execution. Notebooks output is always up to date.

Marimo notebooks: pure Python



Advantages...

- **reactive**
 - ↪ run a cell: marimo reactively runs all dependent cells or marks them as *stale*.
 - ↪ delete a cell: marimo scrubs its variables from program memory, no hidden state.
- **Consistent state**: notebook code, outputs, and program state are guaranteed to be consistent.
- **Reproducible**: no hidden state, deterministic execution. Notebooks output is always up to date.

Marimo notebooks: pure Python



Advantages...

- **reactive**
 - ~> run a cell: marimo reactively runs all dependent cells or marks them as *stale*.
 - ~> delete a cell: marimo scrubs its variables from program memory, no hidden state.
- **Consistent state**: notebook code, outputs, and program state are guaranteed to be consistent.
- **Reproducible**: no hidden state, deterministic execution. Notebooks output is always up to date.

Marimo notebooks: pure Python



Advantages...

- Rich library of UI elements:
 - ↪ Input UI elements: slider, date, number, dropdown, file browser, button, check button, radio button...
 - ↪ Layout UI elements: tab, array, accordion, vstack, hstack...
- UI elements are automatically synchronized with the notebook code:
 - ↪ Interacting with a UI element bound to a global variable automatically runs all cells that reference it.
- and many more...

Marimo notebooks: pure Python



Advantages...

- Rich library of UI elements:
 - ~ Input UI elements: slider, date, number, dropdown, file browser, button, check button, radio button...
 - ~ Layout UI elements: tab, array, accordion, vstack, hstack...
- UI elements are automatically synchronized with the notebook code:
 - ~ Interacting with a UI element bound to a global variable automatically runs all cells that reference it.
- and many more...

Marimo notebooks: pure Python



Advantages...

- Rich library of UI elements:
 - ~ Input UI elements: slider, date, number, dropdown, file browser, button, check button, radio button...
 - ~ Layout UI elements: tab, array, accordion, vstack, hstack...
- UI elements are automatically synchronized with the notebook code:
 - ~ Interacting with a UI element bound to a global variable automatically runs all cells that reference it.
- and many more...

Marimo notebooks: pure Python



Advantages...

- Rich library of UI elements:
 - ~ Input UI elements: slider, date, number, dropdown, file browser, button, check button, radio button...
 - ~ Layout UI elements: tab, array, accordion, vstack, hstack...
- UI elements are automatically synchronized with the notebook code:
 - ~ Interacting with a UI element bound to a global variable automatically runs all cells that reference it.
- and many more...

Marimo notebooks: pure Python



Advantages...

- Rich library of UI elements:
 - ~ Input UI elements: slider, date, number, dropdown, file browser, button, check button, radio button...
 - ~ Layout UI elements: tab, array, accordion, vstack, hstack...
- UI elements are automatically synchronized with the notebook code:
 - ~ Interacting with a UI element bound to a global variable automatically runs all cells that reference it.
- and many more... 😊

Marimo notebooks: pure Python



disadvantages...

- A variable (object) cannot be defined twice in different cells
 - ~ use the format `_name` to make the variable local to the cell
 - ~ or encapsulate object definitions in functions to make them local
- Cell outputs is not written in the notebook (pure Python...) *(not implemented)*
 - ~ The entire notebook is run at startup...
- At startup automatic run of CPU-hungry cells can be problematic:
 - ~ *user setting* ► disable notebook autorun at startup (*lazy notebook*)
- Updating a cell or UI widget triggers the automatic execution of all dependent cells:
 - ~ *user setting* ► disable cell auto run on cell change (*lazy cell*):
marimo will just mark affected cells as *stale*

Marimo notebooks: pure Python



disadvantages...

- A variable (object) cannot be defined twice in different cells
 - ~ use the format `_name` to make the variable local to the cell
 - ~ or encapsulate object definitions in functions to make them local
- Cell outputs is not written in the notebook (pure Python...)
 - ~ The entire notebook is run at startup...
- At startup automatic run of CPU-hungry cells can be problematic:
 - ~ *user setting* ► disable notebook autorun at startup (*lazy notebook*)
- Updating a cell or UI widget triggers the automatic execution of all dependent cells:
 - ~ *user setting* ► disable cell auto run on cell change (*lazy cell*):
marimo will just mark affected cells as *stale*

Marimo notebooks: pure Python



disadvantages...

- A variable (object) cannot be defined twice in different cells
 - ~ use the format `_name` to make the variable local to the cell
 - ~ or encapsulate object definitions in functions to make them local
- Cell outputs is not written in the notebook (pure Python...) *(faded)*
 - ~ The entire notebook is run at startup... *(faded)*
- At startup automatic run of CPU-hungry cells can be problematic: *(faded)*
 - ~ *user setting* ► disable notebook autorun at startup (*lazy notebook*)
- Updating a cell or UI widget triggers the automatic execution of all dependent cells: *(faded)*
 - ~ *user setting* ► disable cell auto run on cell change (*lazy cell*):
marimo will just mark affected cells as *stale*

Marimo notebooks: pure Python



disadvantages...

- A variable (object) cannot be defined twice in different cells
 - ~ use the format `_name` to make the variable local to the cell
 - ~ or encapsulate object definitions in functions to make them local
- Cell outputs is not written in the notebook (pure Python...)
 - ~ The entire notebook is run at startup...
- At startup automatic run of CPU-hungry cells can be problematic:
 - ~ *user setting* ► disable notebook autorun at startup (*lazy notebook*)
- Updating a cell or UI widget triggers the automatic execution of all dependent cells:
 - ~ *user setting* ► disable cell auto run on cell change (*lazy cell*):
marimo will just mark affected cells as *stale*

Marimo notebooks: pure Python



disadvantages...

- A variable (object) cannot be defined twice in different cells
 - ~ use the format `_name` to make the variable local to the cell
 - ~ or encapsulate object definitions in functions to make them local
- Cell outputs is not written in the notebook (pure Python...)
 - ~ The entire notebook is run at startup... 🤔
- At startup automatic run of CPU-hungry cells can be problematic:
 - ~ *user setting* ► disable notebook autorun at startup (*lazy notebook*)
- Updating a cell or UI widget triggers the automatic execution of all dependent cells:
 - ~ *user setting* ► disable cell auto run on cell change (*lazy cell*):
marimo will just mark affected cells as *stale*

Marimo notebooks: pure Python



disadvantages...

- A variable (object) cannot be defined twice in different cells
 - ~ use the format `_name` to make the variable local to the cell
 - ~ or encapsulate object definitions in functions to make them local
- Cell outputs is not written in the notebook (pure Python...)
 - ~ The entire notebook is run at startup... 🤔
- At startup automatic run of CPU-hungry cells can be problematic:
 - ~ *user setting* ▶ disable notebook autorun at startup (*lazy notebook*)
- Updating a cell or UI widget triggers the automatic execution of all dependent cells:
 - ~ *user setting* ▶ disable cell auto run on cell change (*lazy cell*):
marimo will just mark affected cells as *stale*

Marimo notebooks: pure Python



disadvantages...

- A variable (object) cannot be defined twice in different cells
 - ~ use the format `_name` to make the variable local to the cell
 - ~ or encapsulate object definitions in functions to make them local
- Cell outputs is not written in the notebook (pure Python...)
 - ~ The entire notebook is run at startup... 🤔
- At startup automatic run of CPU-hungry cells can be problematic:
 - ~ *user setting* ► disable notebook autorun at startup (*lazy notebook*)
- Updating a cell or UI widget triggers the automatic execution of all dependent cells:
 - ~ *user setting* ► disable cell auto run on cell change (*lazy cell*):
marimo will just mark affected cells as *stale*

Marimo notebooks: pure Python



disadvantages...

- A variable (object) cannot be defined twice in different cells

Editor

Display

Package Management

Runtime

AI

Optional Dependencies

Labs

Version: 0.13.15

RUNTIME CONFIGURATION

Default SQL output Auto (Default) ▾

The default SQL output format for new notebooks; overridden by "sql_output" in the application config.

Autorun on startup ☒

Whether to automatically run all cells on startup.

On cell change autorun ▾

Whether marimo should automatically run cells or just mark them as stale. If "autorun", marimo will automatically run affected cells when a cell is run or a UI element is interacted with; if "lazy", marimo will mark affected cells as stale but won't re-run them.

On module change off ▾

Whether marimo should automatically reload modules before executing cells. If "lazy", marimo will mark cells affected by module modifications as stale; if "autorun", affected cells will be automatically re-run.

Autorun Unit Tests ☒

Enable reactive pytest tests in notebook. When a cell contains only test functions (test_*) and classes (Test_*), marimo will automatically run them with pytest (requires notebook restart).

Learn more in the [docs](#).

Marimo notebooks: pure Python



disadvantages...

- A variable (object) cannot be defined twice in different cells
 - ~ use the format `_name` to make the variable local to the cell
 - ~ or encapsulate object definitions in functions to make them local
- Cell outputs is not written in the notebook (pure Python...)
 - ~ The entire notebook is run at startup... 🤔
- At startup automatic run of CPU-hungry cells can be problematic:
 - ~ *user setting* ► disable notebook autorun at startup (*lazy notebook*)
- Updating a cell or UI widget triggers the automatic execution of all dependent cells:
 - ~ *user setting* ► disable cell auto run on cell change (*lazy cell*):
marimo will just mark affected cells as *stale*

Marimo notebooks: pure Python



disadvantages...

- A variable (object) cannot be defined twice in different cells
 - ~ use the format `_name` to make the variable local to the cell
 - ~ or encapsulate object definitions in functions to make them local
- Cell outputs is not written in the notebook (pure Python...)
 - ~ The entire notebook is run at startup... 🤔
- At startup automatic run of CPU-hungry cells can be problematic:
 - ~ *user setting* ► disable notebook autorun at startup (*lazy notebook*)
- Updating a cell or UI widget triggers the automatic execution of all dependent cells:
 - ~ *user setting* ► disable cell auto run on cell change (*lazy cell*):
marimo will just mark affected cells as *stale*

Marimo notebooks: pure Python



disadvantages...

- A variable (object) cannot be defined twice in different cells
 - ~ use the format `_name` to make the variable local to the cell
 - ~ or encapsulate object definitions in functions to make them local
- Cell outputs is not written in the notebook (pure Python...)
 - ~ The entire notebook is run at startup... 🤔
- At startup automatic run of CPU-hungry cells can be problematic:
 - ~ *user setting* ► disable notebook autorun at startup (*lazy notebook*)
- Updating a cell or UI widget triggers the automatic execution of all dependent cells:
 - ~ *user setting* ► disable cell auto run on cell change (*lazy cell*):
marimo will just mark affected cells as *stale*

Marimo notebooks: pure Python



disadvantages...

- A variable (object) cannot be defined twice in different cells

Runtime Configuration

Default SQL output: Auto (Default) ▾
The default SQL output format for new notebooks; overridden by "sql_output" in the application config.

Autorun on startup ☒
Whether to automatically run all cells on startup

On cell change: autorun ▾
Whether marimo should automatically run cells or just mark them as stale. If "autorun", marimo will automatically run cells when a cell is run or a UI element is interacted with; if "lazy", marimo will mark affected cells as stale and won't re-run them.

On module change: off ▾
Whether marimo should automatically reload modules before executing cells. If "lazy", marimo will mark cells affected by module modifications as stale; if "autorun", affected cells will be automatically re-run.

Autorun Unit Tests ☒
Enable reactive pytest tests in notebook. When a cell contains only test functions (test_*) and classes (Test_*), marimo will automatically run them with pytest (requires notebook restart).

Learn more in the [docs](#).

Version: 0.13.15

Marimo notebooks: pure Python



disadvantages...

- A variable (object) cannot be defined twice in different cells
 - ~ use the format `_name` to make the variable local to the cell
 - ~ or encapsulate object definitions in functions to make them local
- Cell outputs is not written in the notebook (pure Python...)
 - ~ The entire notebook is run at startup... 🤔
- At startup automatic run of CPU-hungry cells can be problematic:
 - ~ *user setting* ► disable notebook autorun at startup (*lazy notebook*)
- Updating a cell or UI widget triggers the automatic execution of all dependent cells:
 - ~ *user setting* ► disable cell auto run on cell change (*lazy cell*):
marimo will just mark affected cells as *stale*

Marimo installation & run

- marimo installation with **uv** (see docs.astral.sh/uv/)

```
uv add marimo           # installs the marimo module within the virt. env.  
uv run marimo edit      # runs marimo editor within your favorite web browser
```

- marimo installation with **pip**

```
pip install marimo  
marimo edit
```

- marimo installation with **conda** (universal virtualenv)

```
conda install -c conda-forge marimo  
marimo edit
```



Marimo notebooks examples

- 1-simple.py
- 2-reactive.py
- 3-interactive.py
- 4-interactive-plot.py
- 5-ML.py
- 5-ML_diskcache.py

Bibliography I

Jupyter notebooks pros/cons...

- [Gru18] J. Grus, “I don’t like notebook,” Allen Institute for Artificial Intelligence. (2018), <https://www.youtube.com/watch?v=7jiPeIFXb6U> (cit. on pp. 4, 5).
- [Guz20] A. Guzharina, “We downloaded 10,000,000 jupyter notebooks from github – this is what we learned,” (2020), <https://blog.jetbrains.com/datalore/2020/12/17/we-downloaded-10-000-000-jupyter-notebooks-from-github-this-is-what-we-learned/#consistency-of-notebooks> (cit. on pp. 13–18).
- [How20] J. Howard, “I like notebook,” fast.ai. (2020), <https://www.youtube.com/watch?v=9Q6sLbz37gk> (cit. on pp. 4, 5).

Marimo Notebooks

- [Agr24] A. Agrawal, “Lessons learned reinventing the python notebook,” marimo.io. (May 8, 2024), <https://marimo.io/blog/lessons-learned?ref=blog.ippon.fr>.
- [Vey24] S. Vey, “Marimo, ou l’avenir du notebook,” ippon positive technology. (June 26, 2024), <https://blog.ippon.fr/2024/06/26/marimo-ou-l-avenir-du-notebook/> (cit. on p. 27).